Week 8: Database Relationship Analysis using Joins

Aim

To retrieve and analyse data from related tables using INNER, LEFT and RIGHT joins, multi-table joins, aggregate functions with GROUP BY, ORDER BY, and subqueries.

Concepts Covered

- **INNER JOIN** returns only rows that match in both tables.
- **LEFT JOIN** returns all rows from the left table, with NULLs where there is no match.
- **RIGHT JOIN** returns all rows from the right table, with NULLs where there is no match.
- **Multi-table JOIN** chains several tables in one query.
- **Aggregates** (SUM, COUNT) with GROUP BY summarise data per group.
- **ORDER BY** sorts the result set.
- **Correlated subquery** is a subquery that refers to the outer query (used with MAX).
- **Inline view with ROWNUM** is the Oracle way to get "Top-N" results.

1. Customers with their orders (INNER JOIN)

```
SQL> SELECT C.C_NAME,
  2         O.ORDER_ID,
  3         O.TOTAL_AMOUNT
  4  FROM CUSTOMER C
  5  INNER JOIN ORDERS O
  6  ON C.CUSTOMER_ID = TO_NUMBER(SUBSTR(O.CUSTOMER_ID, 2));

C_NAME               ORDER_ID TOTAL_AMOUNT
-------------------- -------- ------------
DEVI                 0201              750
DIVI                 0202             1000
KAVYA                0203              500
```

2. Orders with payment status (INNER JOIN)

```
SQL> SELECT O.ORDER_ID,
  2         O.TOTAL_AMOUNT,
  3         P.PAYMENT_STATUS
  4  FROM ORDERS O
  5  INNER JOIN PAYMENT P
  6  ON O.ORDER_ID = P.ORDER_ID;

ORDER_ID TOTAL_AMOUNT PAYMENT_STATUS
-------- ------------ ------------------------
0201              750 Pending
0203              500 Completed
0202             1000 Completed
0204              750 Completed
0205             1000 Completed
0206             1200 Completed
0207              700 Completed
0208             1900 Failed

8 rows selected.
```

3. Customers with no orders (LEFT JOIN ... IS NULL)

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.C_NAME
  3  FROM CUSTOMER C
  4  LEFT JOIN ORDERS O
  5  ON TO_CHAR(C.CUSTOMER_ID) = O.CUSTOMER_ID
  6  WHERE O.ORDER_ID IS NULL;

CUSTOMER_ID C_NAME
----------- --------------------
        101 Arun
        203 KAVYA
        102 Priya
        201 DEVI
        105 Karthik
        103 Kumar
        104 Divya
        202 DIVI

8 rows selected.
```

4. All products with their orders (LEFT JOIN)

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         OD.ORDER_ID
  4  FROM PRODUCT P
  5  LEFT JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID;

PRODUCT_ID PRODUCT_NAME               ORDER_ID
---------- ------------------------- --------
       101  RESIN KEYCHAIN           0201
       102 RESIN BOOKMARK            0202
       103 Resin Pendant             0203
       206 Resin Art Keychain        0204
       207 Resin Art Coaster Set     0205
       208 Resin Art Pendant         0206
       209 Resin Art Photo Frame     0201
       210 Resin Art Bookmark        0202

8 rows selected.
```

5. All orders with customer details (RIGHT JOIN)

```
SQL> SELECT O.ORDER_ID,
  2         O.CUSTOMER_ID,
  3         C.C_NAME,
  4         O.ORDER_DATE,
  5         O.TOTAL_AMOUNT
  6  FROM CUSTOMER C
  7  RIGHT JOIN ORDERS O
  8  ON TO_CHAR(C.CUSTOMER_ID) = O.CUSTOMER_ID;

ORDER_ID CUSTOMER C_NAME               ORDER_DAT TOTAL_AMOUNT
-------- -------- -------------------- --------- ------------
0205     C205                          26-AUG-26         1000
0208     C208                          29-AUG-26         1900
0207     C207                          28-AUG-26          700
0203     C203                          24-AUG-26          500
0206     C206                          27-AUG-26         1200
0202     C202                          23-AUG-26         1000
0201     C201                          22-AUG-26          750
0204     C204                          25-AUG-26          750

8 rows selected.
```

6. Orders with no matching customer (RIGHT JOIN ... IS NULL)

```
SQL> SELECT O.ORDER_ID,
  2         O.CUSTOMER_ID,
  3         C.C_NAME
  4  FROM CUSTOMER C
  5  RIGHT JOIN ORDERS O
  6  ON TO_CHAR(C.CUSTOMER_ID) = O.CUSTOMER_ID
  7  WHERE C.CUSTOMER_ID IS NULL;

ORDER_ID CUSTOMER C_NAME
-------- -------- --------------------
0205     C205
0208     C208
0207     C207
0203     C203
0206     C206
0202     C202
0201     C201
0204     C204

8 rows selected.
```

7. Payments with order details (RIGHT JOIN)

```
SQL> SELECT P.PAYMENT_ID,
  2         P.ORDER_ID,
  3         P.PAYMENT_MODE,
  4         P.PAYMENT_STATUS,
  5         O.ORDER_DATE,
  6         O.TOTAL_AMOUNT
  7  FROM ORDERS O
  8  RIGHT JOIN PAYMENT P
  9  ON O.ORDER_ID = P.ORDER_ID;

PAYMENT_ID ORDER_ID PAYMENT_MODE         PAYMENT_STATUS           ORDER_DAT TOTAL_AMOUNT
---------- -------- -------------------- ------------------------ --------- ------------
         1 0201     UPI                  Pending                  22-AUG-26          750
         2 0202     Credit Card          Completed                23-AUG-26         1000
         3 0203     Debit Card           Completed                24-AUG-26          500
         4 0204     Net Banking          Completed                25-AUG-26          750
         5 0205     UPI                  Completed                26-AUG-26         1000
         6 0206     Credit Card          Completed                27-AUG-26         1200
         7 0207     Debit Card           Completed                28-AUG-26          700
         8 0208     Debit Card           Failed                   29-AUG-26         1900

8 rows selected.
```

8. Customer, product and order (multi-table JOIN)

```
SQL> SELECT C.C_NAME,
  2         P.PRODUCT_NAME,
  3         OD.ORDER_ID
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  7  JOIN ORDER_DETAILS OD
  8      ON O.ORDER_ID = OD.ORDER_ID
  9  JOIN PRODUCT P
 10      ON OD.PRODUCT_ID = P.PRODUCT_ID;

C_NAME               PRODUCT_NAME              ORDER_ID
-------------------- ------------------------- --------
DEVI                  RESIN KEYCHAIN           0201
DIVI                 RESIN BOOKMARK            0202
KAVYA                Resin Pendant             0203
DEVI                 Resin Art Photo Frame     0201
DIVI                 Resin Art Bookmark        0202
```

9. Total spent per customer (SUM + GROUP BY)

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.C_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  7  GROUP BY C.CUSTOMER_ID, C.C_NAME;

CUSTOMER_ID C_NAME               TOTAL_SPENT
----------- -------------------- -----------
        202 DIVI                        1000
        203 KAVYA                        500
        201 DEVI                         750
```

10. Number of orders per customer (COUNT + LEFT JOIN)

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.C_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  7  GROUP BY C.CUSTOMER_ID, C.C_NAME
  8  ORDER BY C.CUSTOMER_ID;

CUSTOMER_ID C_NAME               TOTAL_ORDERS
----------- -------------------- ------------
        101 Arun                            0
        102 Priya                           0
        103 Kumar                           0
        104 Divya                           0
        105 Karthik                         0
        201 DEVI                            1
        202 DIVI                            1
        203 KAVYA                           1

8 rows selected.
```

11. Each customer's latest order (correlated subquery)

```
SQL> SELECT C.C_NAME,
  2         O.ORDER_ID,
  3         O.ORDER_DATE,
  4         O.TOTAL_AMOUNT
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  8  WHERE O.ORDER_DATE = (
  9      SELECT MAX(O2.ORDER_DATE)
 10      FROM ORDERS O2
 11      WHERE O2.CUSTOMER_ID = O.CUSTOMER_ID
 12  );

C_NAME               ORDER_ID ORDER_DAT TOTAL_AMOUNT
-------------------- -------- --------- ------------
DEVI                 0201     22-AUG-26          750
DIVI                 0202     23-AUG-26         1000
KAVYA                0203     24-AUG-26          500
```

12. Orders sorted by date (ORDER BY)

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.C_NAME,
  3         O.ORDER_ID,
  4         O.ORDER_DATE,
  5         O.TOTAL_AMOUNT,
  6         O.ORDER_STATUS
  7  FROM CUSTOMER C
  8  JOIN ORDERS O
  9  ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
 10  ORDER BY O.ORDER_DATE;

CUSTOMER_ID C_NAME               ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
----------- -------------------- -------- --------- ------------ ------------------------
        201 DEVI                 0201     22-AUG-26          750 Pending
        202 DIVI                 0202     23-AUG-26         1000 Shipped
        203 KAVYA                0203     24-AUG-26          500 Delivered
```

13. Total quantity sold per product

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID PRODUCT_NAME              TOTAL_QUANTITY_SOLD
---------- ------------------------- -------------------
       103 Resin Pendant                               4
       210 Resin Art Bookmark                          3
       102 RESIN BOOKMARK                              3
       208 Resin Art Pendant                           2
       101  RESIN KEYCHAIN                             2
       206 Resin Art Keychain                          2
       207 Resin Art Coaster Set                       1
       209 Resin Art Photo Frame                       1

8 rows selected.
```

14. Payment table listing

```
SQL> SELECT PAYMENT_ID,
  2         ORDER_ID,
  3         PAYMENT_MODE,
  4         PAYMENT_STATUS,
  5         TRANSACTION_AMOUNT
  6  FROM PAYMENT
  7  ORDER BY PAYMENT_ID;

PAYMENT_ID ORDER_ID PAYMENT_MODE         PAYMENT_STATUS            TRANSACTION_AMOUNT
---------- -------- -------------------- ------------------------ ------------------
         1 0201     UPI                  Pending                             750
         2 0202     Credit Card          Completed                          1000
         3 0203     Debit Card           Completed                           500
         4 0204     Net Banking          Completed                          7000
         5 0205     UPI                  Completed                          5200
         6 0206     Credit Card          Completed                          2450
         7 0207     Debit Card           Completed                         20000
         8 0208     Debit Card           Failed                            26000

8 rows selected.
```

15. Top 3 customers by spending (inline view + ROWNUM)

```
SQL> SELECT *
  2  FROM (
  3       SELECT C.CUSTOMER_ID,
  4              C.C_NAME,
  5              SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  6       FROM CUSTOMER C
  7       JOIN ORDERS O
  8       ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  9       GROUP BY C.CUSTOMER_ID, C.C_NAME
 10       ORDER BY TOTAL_SPENT DESC
 11  )
 12  WHERE ROWNUM <= 3;

CUSTOMER_ID C_NAME               TOTAL_SPENT
----------- -------------------- -----------
        202 DIVI                        1000
        201 DEVI                         750
        203 KAVYA                        500
```

16. Customer with the most orders

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.C_NAME,
  5             COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  6      FROM CUSTOMER C
  7      LEFT JOIN ORDERS O
  8      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  9      GROUP BY C.CUSTOMER_ID, C.C_NAME
 10      ORDER BY TOTAL_ORDERS DESC
 11  )
 12  WHERE ROWNUM = 1;

CUSTOMER_ID C_NAME               TOTAL_ORDERS
----------- -------------------- ------------
        202 DIVI                            1
```

17. Highest-spending customer

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.C_NAME,
  5             SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON TO_CHAR(C.CUSTOMER_ID) = SUBSTR(O.CUSTOMER_ID, 2)
  9      GROUP BY C.CUSTOMER_ID, C.C_NAME
 10      ORDER BY TOTAL_SPENT DESC
 11  )
 12  WHERE ROWNUM = 1;

CUSTOMER_ID C_NAME               TOTAL_SPENT
----------- -------------------- -----------
        202 DIVI                        1000
```

Conclusion

The experiment demonstrated how to retrieve related data using joins, summarise it with aggregate functions and subqueries, and produce Top-N results in Oracle SQL. It also showed that data-type consistency between key columns is essential for correct results.